

Overview

In this hands-on workshop, you'll learn how to create and configure specialized AI agents that enhance your development workflow. By the end of this lab, you'll understand how to build agents tailored to specific projects, roles, and use cases.

Learning Objectives

By completing this lab, you will be able to:

- Configure agent behavior using prompts and context
- Integrate external tools via Model Context Protocol (MCP)
- Implement security controls and tool permissions
- Set up project-specific and global agents
- Use lifecycle hooks for automation
- Manage legacy compatibility settings

Prerequisites:

- Kiro CLI installed and authenticated
- **uv (Python package installer)** - [Installation Guide](#) 
- **AWS credentials configured** (for pricing MCP server - API calls are free)
- Basic understanding of JSON configuration
- Familiarity with command-line interfaces

Custom Agent

Agent Configuration Structure

```
your-project/
├── .kiro/
│   └── agents/          ← Project-specific agents
│       ├── frontend-dev.json
│       ├── backend-api.json
│       └── devops-specialist.json
├── src/
├── docs/
└── README.md

~/.kiro/agents/          ← Global agents
├── general-dev.json
└── security-auditor.json
```

Configuration Priority

1. **Project-specific:** `~/.kiro/agents/*.json` (highest priority)
2. **Global:** `~/.kiro/agents/*.json` (fallback)

Configuration Areas to Be Explored

1. Agent Prompts

What you'll learn: How to define agent personality and behavior

```
{
  "prompt": "You are a senior React developer specializing in modern hooks and performance optimization"
}
```

Lab Activities:

- Create role-specific prompts for different development personas
- Configure context-aware behavior for project domains
- Test prompt effectiveness with real scenarios

2. MCP Server Integration

What you'll learn: Connect external tools and services to your agents

```
{
  "mcpServers": {
    "aws-knowledge-mcp-server": {
      "command": "uvx",
      "args": [
        "mcp-proxy",
        "--transport",
        "streamablehttp",
        "https://knowledge-mcp.global.api.aws"
      ]
    },
    "awslabs.aws-pricing-mcp-server": {
      "command": "uvx",
      "args": ["awslabs.aws-pricing-mcp-server@latest"],
      "env": {
        "FASTMCP_LOG_LEVEL": "ERROR",
        "AWS_REGION": "ap-southeast-1"
      }
    }
  }
}
```

Lab Activities:

- Set up AWS Knowledge Base integration for documentation access
- Configure AWS Pricing server for cost analysis
- Create AWS-focused agent configurations

3. Tools Configuration

What you'll learn: Control which tools agents can access and how they behave

```
{
  "tools": ["read", "write", "@aws-knowledge-mcp-server", "@awslabs.aws-pricing-mcp-server"],
  "allowedTools": ["read"],
  "toolAliases": {
    "@aws-knowledge-mcp-server/aws__search_documentation": "aws_search",
    "@awslabs.aws-pricing-mcp-server/get_pricing": "pricing_lookup"
  }
}
```

Lab Activities:

- Configure available tools for different agent roles
- Set up auto-approved tools for smooth workflows
- Create tool aliases to avoid naming conflicts
- Implement security restrictions with allowed/denied patterns
- Integrate AWS Knowledge Base and Pricing tools

4. Context Resources

What you'll learn: Provide agents with project-specific knowledge

```
{
  "resources": [
    "file://README.md",
    "file://docs/architecture.md",
    "file://src/types/index.ts"
  ]
}
```

Lab Activities:

- Include project documentation in agent context
- Set up dynamic resource patterns with wildcards
- Configure environment-specific context files

5. Lifecycle Hooks

What you'll learn: Automate tasks when agents start or receive prompts

```
{
  "hooks": {
    "agentSpawn": [{"command": "echo 'Development agent ready'"}],
    "userPromptSubmit": [{"command": "log-interaction.sh"}]
  }
}
```

Lab Activities:

- Create startup automation with agentSpawn hooks
- Implement logging and monitoring with userPromptSubmit hooks
- Build custom workflow automation scripts

6. Legacy Compatibility

What you'll learn: Integrate with existing MCP configurations

```
{
  "useLegacyMcpJson": true
}
```

Lab Activities:

- Understand legacy MCP integration
- Migrate from global MCP configurations
- Handle configuration conflicts and priorities

Lab Summary

- **Tools configuration** (`tools`) to control agent capabilities
- **Allowed tools** (`allowedTools`) for automatic approval and workflow efficiency
- **Agent prompts** (`prompt`) for role-specific behavior
- **Context resources** (`resources`) for project-specific knowledge
- **Tools settings** (`toolsSettings`) with security controls and permissions
- **MCP integrations** (`mcpServers`) for external tool access
- **Tool aliases** (`toolAliases`) for improved usability
- **Lifecycle hooks** (`hooks`) for automation
- **Complete integration** with production-ready configurations

Let's begin! 🚀🔗

[Previous](#)[Next](#)